

팀원 온보딩 — 속도는벡터 캡스톤 (2026-04-17 작성 · 2026-04-19 10축 ultrareview 반영)

독자: 강재현·박세은·이동욱. 조현빈이 Claude Code 와 함께 4월 14일~17일 동안 진행한 실험·보고서·발표 자료의 맥락을 빠르게 따라잡기 위해 작성한 문서이며, 4월 19일 10축 극한 디리뷰 (CRITICAL 6 + HIGH 26 해소) 를 반영하여 본 v2 로 갱신되었습니다. **읽는 방식:** §0만 읽으면 5분, 전체는 약 25분. §4·§5·§7은 발표 질의응답 대비에 직접 씁니다. **다음 마감:** 2026-04-28 중간발표 + 중간보고서 제출 (주 발표자 강재현). **근거 문서:** 모든 수치는 [submission/속도는벡터_중간보고서_20260417_0000.md](#) , [experiments/results/RQ1_RQ2 실험 결과 정리.md](#) , [plans/RQ3설계안_20260416_213500.md](#) 에서 그대로 가져왔습니다. 의심스러운 숫자가 있으면 이 세 문서를 원본으로 보십시오.

§0. 3분 요약 — 바쁠 때 이것만

한 문장 정리

우리는 Exqutor 라는 벡터 DB 카디널리티 추정 논문의 **단일 테이블 사각지대**를 발견했고, 데이터의 공간 구조를 아는 **총화 샘플링(KM20)** 으로 추정 정확도를 개선했으며, DEEP·SIFT 두 데이터셋 3종 규모(1M·8M·1.5M)에서 5-seed 95% 신뢰구간으로 효과가 재현됨을 확인했습니다.

핵심 숫자 다섯 개

지표	값	의미
DEEP 1M s=1% KM20 vs RANDOM20 격차	19.6%p	공간 구조를 아는 분할이 모르는 분할보다 19.6%p 더 정확 (s=0.010에서)
SIFT 1.5M s=50% KM20 개선	+3.07% CI [+2.66, +3.48]	DEEP의 +1.64% 대비 2배 — 더 쏠린 데이터일수록 이점이 큼
DEEP 8M s=50% KM20 개선	+1.76% CI [+0.65, +2.86]	1M의 [+1.11, +2.18]과 CI 겹쳐 CONSISTENT — 8배 규모에서도 재현
Two-Level Decomposition L2 (s=1%)	+19.60%p	개선의 원인이 "20등분" 이 아니라 "공간 구조 얇" 임을 수학적으로 분리
SIFT CV vs DEEP CV	0.394 vs 0.234 (68%↑)	쏠림 정도의 정량 차이가 효과 격차 2배의 직접 원인

데이터셋별 KM20 개선을 한눈에 비교

교수·팀 보고용 원페이지 표. BERN 대비 median Q-error 개선을 (5-seed 평균 [95% CI]).

데이터셋	차원	규모	s=50%	s=5%	s=1%	쏠림 (CV)
DEEP 1M	96d	100만	+1.64% [+1.11, +2.18]	+1.85% (단일)	+8.93% (단일)	0.234
DEEP 8M	96d	800만	+1.76% [+0.65, +2.86]	+0.55% [-3.11, +4.21]	노이즈 지배	0.234
SIFT 1.5M	128d	150만	+3.07% [+2.66, +3.48]	+4.39% [+2.63, +6.15]	-0.53% (양자화)	0.394

해독 포인트:

- **s=50% (넓은 쿼리):** DEEP 1M/8M 이 +1.64~1.76% 로 CONSISTENT (규모 8배 차이에도 재현), SIFT 가 +3.07% 로 CI 분리되며 약 **2배**.
- **s=5%:** DEEP 효과 작음, SIFT +4.39% 로 확연히 큼 — 쏠림 격차가 중간대 selectivity 에서 가장 두드러짐.
- **s=1%:** DEEP 1M 만 +8.93% 큰 개선. DEEP 8M 과 SIFT 는 sample_size=385 한계로 노이즈/양자화 — 안정 하한 $s \geq 0.050$ boundary condition.
- **쏠림 (CV):** SIFT 가 DEEP 대비 68% 더 쏠림 — **효과 2배의 직접 원인** (상관이 아니라 인과).

한 줄 요약: 더 쏠린 데이터셋(SIFT)에서 공간 인식 샘플링의 이점이 더 크다 (CV 68% ↑ → 효과 2배).

지금 위치

- **RQ1(문제 제기):** Exqutor 단일 테이블 사각지대 + 8지표 부정 결과  완료

- RQ2(분포 인지 개선): KM20 stratified 네이티브 구현 + 5-seed CI + Two-Level 분해  완료
- RQ3(분포 무인지 회수): 7가지 방법 × 3 패러다임 설계안만 확정, 실험은 4/28 이후

4/28까지 남은 일

1. 팀 온보딩(이 문서) + 4/22 팀 리뷰 + 리허설 2회
2. PPT 발표자 노트 작성 (강재현 본인이 슬라이드별 스크립트 내재화)
3. 중간보고서 교열
4. 자문 회신 반영 (회신 도착 시에만, §2~§4 톤 조정)
5. 서버 추가 실험 없음. 보충 실험 5건은 모두 W5(4/28~) 최종발표 준비로 이관됨

§1. 왜 이 연구인가 — 큰 그림

문제의 뿌리: 벡터 DB의 카디널리티 추정이 왜 어려운가

관계형 DB에서 쿼리 옵티마이저는 "이 조건에 맞는 행이 대략 몇 개일까"를 추정해서 실행 계획을 세웁니다.

`WHERE age = 30` 같은 조건이면 히스토그램으로 쉽게 추정할 수 있습니다. 그런데 벡터 DB에서는 `WHERE (벡터 <-> 쿼리벡터) < 0.5` 같은 조건이 등장합니다. 이건 **거리 기반 조건**이라 히스토그램이 안 통합니다. 기존 시스템들의 대응:

- pgvector: 33.3% 로 고정 (무조건 전체의 1/3)
- VBASE: 50% 로 고정
- DuckDB: 100% 로 고정 (즉 안 걸려진다고 가정)

이 추정이 틀리면 조인 순서가 엉망이 되고, 최악은 수백~수천 배 느려집니다. 실제 selectivity 는 쿼리에 따라 0.001% 부터 90% 까지 변동하므로 고정값은 터무니없는 가정입니다.

Exqutor 의 해법과 그 빈틈

Exqutor (arXiv:2512.09695v2, 2024) 는 이 문제의 최신 해법입니다. 두 축으로 구성:

1. **ECQO**: HNSW 인덱스가 있으면 range query 를 직접 실행해 1~2ms 안에 **정확한** 카디널리티를 얻음. 정답을 구하는 거라 빈틈이 없음.
2. **Adaptive Sampling**: HNSW 가 없으면 데이터 일부를 무작위로 뽑아 조건 맞는 비율을 계산, 모멘텀 기반으로 sample_size 를 조정해 근사. pgvector 에서 최대 1000배, VBASE 에서 10000배, DuckDB 에서 1.5~37 배 성능 향상 보고.

우리의 공략점은 2번 **Adaptive Sampling** 입니다. 무작위 샘플이 정확해지려면 "샘플이 전체 데이터를 대표한다"는 가정이 필요한데, 현실의 벡터 데이터는 한쪽으로 쏠려 있습니다 (skewed). 쏠린 상태에서 무작위 샘플을 뽑으면 밀도 높은 영역이 과대표집되어 카디널리티가 왜곡됩니다.

4/3 교수님 미팅에서 확정된 뼈대

박세은 팀장이 참석한 4/3 교수님 미팅에서 다음 프레이밍이 확정되었습니다. 이게 본 연구의 헌법입니다.

"데이터가 **일반적**(정규분포 같은 균일한 형태)일 때와 **한쪽으로 쏠려있는** 경우 사이의 성능 차이. Exqutor는 쏠린 상황에서도 기존과 **똑같은** uniform sampling 을 사용하니, 그런 케이스에서 **성능이 떨어지고**, 따라서 **샘플링 개선**을 통해서 이를 해결하자."

이 문장을 3단계로 풀면:

Step 1: 데이터가 쏠려있다는 것을 보여준다
Step 2: 쏠려있을 때 uniform sampling 이 실제로 나빠지는 것을 보여준다
Step 3: 개선된 sampling 이 이를 해결하는 것을 보여준다

우리 실험이 전부 이 3단계를 따릅니다. 보고서를 읽을 때 "이게 Step 몇을 뒷받침하는가"를 항상 자문하면 구조가 보입니다.

4/14 에 드러난 두 가지 반전

본격 실험 시작 직후 **두 가지 반전**이 있었습니다. 이걸 이해해야 보고서 §4.2~§4.3 의 논리가 읽힙니다.

반전 1 — 첫 가설(H1) 기각. 처음엔 "쿼리 벡터 근처의 데이터 분포가 skewed 할수록(Fisher γ 같은 지표가 클수록) Q-error 가 크다"고 예측했습니다. 4/14 아침, 100 query $\times$ 6 selectivity $\times$ 5 seed = 3000 측정으로 검증한 결과, 모든 조합에서 Spearman 상관 $|p| < 0.2$ 로 **강하게 기각**. 그날 저녁 로컬 4지표(k-NN entropy, PCA EVR1, KDE modality, NN clustering) 까지 추가 검증했지만 **8지표 전수** 모두 무효. 이유는 고차원(96d) 벡터 특유의 "거리 집중 현상(distance concentration)" — 100개 쿼리가 모두 비슷한 분포를 가져서 지표의 변별력이 평탄해졌습니다.

반전 2 — Exqutor 단일 테이블 사각지대 발견. Python 재구현을 네이티브와 비교하려다 `vector.c` line 243 의 `if (table_count > 2)` 조건을 발견. **단일 테이블** 벡터 쿼리에는 Exqutor 의 Adaptive Sampling 축이 아예 작동하지 않고 PG 기본값 33.3% 로 떨어집니다. 이 한 줄을 `>= 1` 로 우회하면 이번엔 outer query 의 plan tree 가 `Sample Scan` 으로 교체되어 outer query 결과 자체가 부분 카운트로 격하됩니다. Exqutor 논문은 multi-table join 시나리오만 가정했기 때문에 이런 제약이 본문에 명시되어 있지 않았고, 우리가 소스 검증으로 처음 정량 확인했습니다. 이미지 검색·추천 시스템·RAG retrieval 같은 **실무 시나리오가 모두 단일 테이블 쿼리**라는 점에서 이걸 학술적·실무적으로 모두 채울 가치가 있는 빈자리입니다.

두 반전을 재조립해서 만든 피벗(Pivot A + Pivot C)

4/14 17:35 에 다음과 같이 재조립했습니다.

- **시나리오를 좁힌다**: multi-table join → 단일 테이블 vector range query 사각지대. 이쪽이 실무적으로 더 중요하고, Exqutor 가 명시적으로 다루지 않으므로 학술적 빈자리도 큼니다.
- **측정 도구를 바꾼다**: query 기반 skewness 지표는 무효였으므로, 데이터 자체의 공간 구조(k-means 파티션)를 사용합니다.

그 위에 두 가지 수정(sanitize)을 구현:

- **Pivot A**: `vector.c` L889 의 `TABLESAMPLE SYSTEM` → `BERNOULLI` 한 줄 교체. SYSTEM 은 디스크 블록 단위 샘플링이라 같은 블록 행이 묶여 뽑히는 spatial bias 가 있고, BERNOLLI 는 행 단위 독립 샘플링입니다. 이건 **공정한 baseline** 을 만드는 도구이지 단독 기여가 아닙니다.
- **Pivot C**: `vector.c` 에 +228 줄 추가해서 k-means K=20 기반 층화 샘플링을 네이티브로 구현. 이게 본 연구의 핵심 기여입니다.

두 피벗은 직교적이라 `SYSTEM/BERNOULLI/STRATIFIED × dataset × selectivity` 로 ablation matrix 를 만들어 각 기여를 분리 측정할 수 있습니다.

§2. 용어사전 — 읽기 전에 외워둘 것

발표 Q&A 에서 교수님이나 다른 팀이 쓸 법한 용어를 모았습니다. 우리가 정의한 고유 용어는 ★ 표시.

데이터·쿼리 관련

벡터 범위 쿼리 (vector range query): `WHERE (v <-> q) < D` 꼴. v=데이터 벡터, q=쿼리 벡터, D=거리 임계값. 임계값 안에 들어오는 행의 개수(카디널리티)가 추정 대상.

selectivity (s): 전체 데이터 중 조건에 맞는 비율. 우리가 쓰는 5점: 0.010, 0.050, 0.100, 0.300, 0.500 (일부 분석은 0.001 포함). 각 selectivity 의 D_{target} 은 데이터셋별로 quantile 로 사전 계산. **s 가 작을수록 "좁은 쿼리"** 이고 추정이 더 어렵습니다.

D_{target} : 특정 selectivity 를 달성하기 위한 거리 임계값 D. 데이터셋마다 다르게 계산해야 함. 이걸 잘못 재계산 하면 의도한 s 가 실제로 0.0001 같은 극극소로 이동해서 artifact 가 생깁니다 (실제 Phase 7 에서 발생).

DEEP / SIFT: 표준 벡터 벤치마크 데이터셋. DEEP 은 96차원 딥러닝 이미지 특징, SIFT 는 128차원 이미지 키폴인트. **SIFT 가 DEEP 보다 더 쏠려있다** (CV 0.394 vs 0.234).

추정·측정 관련

Q-error: 추정 카디널리티와 실제 카디널리티의 비율. $\max(\text{est}/\text{true}, \text{true}/\text{est})$ 로 1.0 이 완벽, 클수록 나쁨. 우리는 median Q-error 로 비교.

sample_size: Exqutor 가 뽑는 샘플 행 수. 기본 385 (통계적 공식 기반). 이 값이 고정되면 s 가 매우 작을 때 ($s=0.001$ 이면 true_card 가 1000 정도) 샘플 안 매칭이 0~1개밖에 안 나와서 noise 지배.

hook_est: Exqutor 가 서버 로그에 `eelog` 로 남기는 추정값. 우리는 **이걸 authoritative metric** 으로 씁니다. plan_rows 는 신뢰하지 않음 (Phase 7 artifact 원인).

paired Wilcoxon: 같은 100개 쿼리에 대해 두 방식을 쌍으로 비교하는 비모수 검정. $p < 0.05$ 면 "통계적으로 유의한 차이".

5-seed / 95% CI: 같은 실험을 setseed 0.1~0.5 다섯 번 반복. 95% 신뢰구간은 $\text{mean} \pm t(4, 0.975) \times \text{std}/\sqrt{5}$, 여기서 $t = 2.776$. **CI 하한이 양수**이면 real effect 확정.

샘플링 방식

TABLESAMPLE SYSTEM / BERNOULLI: PostgreSQL 샘플링 모드. SYSTEM 은 블록 단위(빠르지만 편향), BERNOULLI 는 행 단위(느리지만 편향 없음).

k-means K=20 (KM20) ★: 데이터 전체를 사전에 20개 클러스터로 분할. 우리 연구의 핵심 파티션.

RANDOM20 ★: 20개 그룹에 완전 무작위로 배정. 공간 구조 무시. KM20 의 대조군.

stratified sampling (층화 샘플링): 각 그룹에서 비례 배분으로 샘플 추출 후 Horvitz-Thompson 가중 추정 $\text{est} = \sum (n_i \cdot \text{cnt}_i / s_i)$.

stratum_id: 각 행에 부여된 클러스터 라벨(0~19). `smallint` 컬럼으로 미리 저장하므로 쿼리 시점 overhead 0.

우리 고유 개념 ★

Pivot A: `TABLESAMPLE SYSTEM → BERNOULLI` 한 줄 교체. fair baseline 도구.

Pivot C: KM20 기반 stratified sampling 네이티브 구현. 본 연구 핵심.

Two-Level Decomposition: - Level 1 (비례 배분) = RANDOM20 의 개선분. "20등분" 자체가 제공하는 표본 크기 안정화. - Level 2 (공간 인식) = KM20 개선 - RANDOM20 개선. 공간 구조를 아는 파티션만의 추가 이점. - Total = Level 1 + Level 2 = KM20 총 개선.

Recovery Rate (RQ3 지표): $(\text{방법X} - \text{RANDOM20}) / (\text{KM20} - \text{RANDOM20})$. 1.0 = KM20 과 동등(oracle), 0.0 = RANDOM20 과 동등, 음수 = RANDOM20 보다 나쁨.

boundary condition ★: 우리가 정량 확정한 경계 — $s \geq 0.050$ 에서만 KM20 효과가 안정. 그 아래는 sample_size=385 한계로 noise 지배.

썰림 정량 지표

HHI (Herfindahl-Hirschman Index): 각 클러스터 비율의 제곱합. 균일이면 $1/K=0.05$, 한 클러스터 집중이면 1.0. DEEP 0.05274 / SIFT 0.05776.

CV (Coefficient of Variation): 표준편차/평균. 균일이면 0, 썰릴수록 커짐. DEEP 0.234 / SIFT 0.394 (68% 차이).

§3. 실험 세팅 — 어디서 어떻게 돌렸는지

서버

- 호스트: `165.132.140.240` (연세대 BDAI Lab)
- 계정: `capstone2026`
- 작업 디렉토리: `/mnt/hdd0/home/capstone2026`
- 임채림 선배가 4/13 에 vanilla_sf100 배치 완료, 4/14 에 우리가 환경 세팅 완료
- 메모리의 `reference_server.md` 에 상세 접속/빌드 정보

두 개의 PostgreSQL 인스턴스

서버에는 PG 인스턴스가 두 개 돌고 있습니다. 혼동 주의:

포트	용도	주요 테이블
55435	vanilla PG 16.9 + pgvector 0.7.1 (HNSW 보유)	<code>partsupp_deep_10</code> , <code>customer_sift_10</code>
55436	Exqutor 확장 PG (우리가 <code>vector.c</code> 수정)	<code>partsupp_deep_10_subset_1m</code> , <code>..._phase7_8m_subset</code> , <code>customer_sift_10_phase7_noidx_subset</code>

Exqutor 측정은 전부 55436 에서 합니다. vanilla 55435 는 baseline HNSW 성능 비교용으로만 사용.

데이터셋 3종

이름	행 수	차원	특성
<code>partsupp_deep_10_subset_1m</code>	100만	96d	TPC-H sf10 기반 DEEP 벡터, 상대적으로 균일
<code>partsupp_deep_10_phase7_8m_subset</code>	800만	96d	위와 같은 종류 8배 규모, 외적 타당성 검증용
<code>customer_sift_10_phase7_noidx_subset</code>	150만	128d	SIFT 벡터, HNSW 없이 별도 transfer (838MB, 2분 33초)

각 테이블에 `stratum_id smallint` 컬럼과 인덱스를 **사전 부여**합니다. k-means K=20, seed=42 로 numpy mini-batch 학습 후 UPDATE.

쿼리 규약

- 100개 쿼리 무작위 추출 (각 데이터셋마다)
- selectivity 5점: 0.010, 0.050, 0.100, 0.300, 0.500 (1% 는 보조 분석)
- 각 (쿼리, selectivity) 조합의 D_target 은 **그 데이터셋에서 quantile** 로 계산 (Phase 7 교훈 후 각 데이터셋 별 재계산)
- setseed 0.1 ~ 0.5 의 5 seed 반복
- 각 seed × 쿼리 × selectivity × mode(BERN/STRAT) 측정
- 결과: per-seed Q-error 시계열 → 5-seed 평균 + 95% CI + paired Wilcoxon p

측정 파이프라인 (재사용 가능)

RQ2 에서 완성된 `multiseed_paired` 파이프라인:

1. Python 에서 D_target numpy 계산 (Exqutor hook 우회)
2. 각 쿼리에 대해 `EXPLAIN (ANALYZE, FORMAT JSON)` 실행
3. 서버 로그에서 `hook_est` 파싱 (plan_rows 는 사용 안 함)
4. Q-error 계산, 5-seed 집계
5. paired Wilcoxon p, 95% CI 산출

RQ3 에서도 이 파이프라인을 그대로 재사용. 각 방법마다 Python 에서 stratum_id 만 다르게 계산하고 DB에 UPDATE 한 뒤 위 파이프라인 실행. H(Importance Sampling) 만 예외 (hook_est 반환 공식 수정).

왜 복잡한가 — 회피해야 했던 세 함정

이런 세팅이 정착되기까지 세 가지 함정에 빠졌습니다 (`reference_server.md` 에도 정리됨):

1. Exqutor hook 이 SIFT range query 에서 크래시 — `postgresql.conf` 에 `update_sample_size=off` 를 기본값으로 추가해야 함. Exqutor 의 Adaptive update 경로 자체가 multi-table 전제라 단일 테이블에서 SIGSEGV 납니다.
2. D_target binary search 에서 Exqutor hook 개입 — SQL 기반 binary search 를 쓰면 Exqutor 가 중간에 간섭. numpy 로 우회.
3. 로그 파싱 offset 불일치 — PG 재시작으로 로그가 리셋되는 경우 offset 기반 파싱이 엉망. 안정 환경에서 재 실행 규약.

§4. 결과 해독법 — 핵심 표 다섯 개 줄별 해설

보고서 §4 와 발표 Slide 8~11 의 핵심 표 5개를 "이 숫자가 무슨 의미인지" 관점에서 풀어씁니다.

표 1 — Pivot A (SYSTEM vs BERNOULLI, DEEP 1M)

selectivity	SYS median	BERN median	diff%	p	유의
0.001	2.597	2.597	+0.0	0.596	—
0.010	1.558	1.299	+20.0	0.240	—
0.050	1.203	1.195	+0.7	0.0009	★★
0.100	1.212	1.132	+7.0	<0.001	★★★
0.300	1.210	1.079	+12.0	<0.001	★★★
0.500	1.153	1.052	+9.6	<0.001	★★★

- 이 표는 뭘 보여주는가: Exqutor 의 기본 `TABLESAMPLE SYSTEM` (블록 단위)을 `BERNOULLI` (행 단위)로 한 줄 교체했을 때 중간~넓은 selectivity 전부에서 Q-error 가 유의하게 개선됨.
- diff% 는 $\text{median}(\text{SYS}) - \text{median}(\text{BERN})$ 을 $\text{median}(\text{SYS})$ 로 나눈 상대 개선율. 즉 BERN 이 SYS 보다 얼마나 작은 Q-error 를 냈는지. 4 구간에서 $p < 0.001$ ($s=0.05$ 만 $p=0.0009$) — block sampling bias 가 실존하는 구조적 약점임이 네이티브에서 증명.
- 왜 $s=0.001$ 에선 동률인가: sample 385 안에 매칭이 0~1 개만 뽑혀서 BERN 의 `cnt=0 → 1` clamp fallback 과 SYS 양쪽 다 같은 값을 반환. 이 영역은 sampling method 와 무관하게 구조적으로 변별 불가능. 이걸 버그가 아니라 boundary condition.
- 발표 포인트: "한 줄 교체"가 드라마틱하지만 이건 단독 기여가 아닌 fair baseline 도구. 진짜 본 연구 기여는 아래 Pivot C.

표 2 — Pivot C (KM20) 5-seed CI, DEEP 1M

selectivity	5-seed 평균	95% CI	유의한 seed
50%	+1.64%	[+1.11, +2.18]	5/5
30%	+2.62%	[+1.45, +3.80]	5/5 (p<0.001)
10%	+4.19%	[+0.72, +7.66]	5/5 (p<0.004)
5%	+1.85%	(단일 측정)	3/5
1%	+8.93%	(단일 측정)	4/5

- **이 표가 왜 중요한가:** 처음 Phase 6 Step 4 에서 단일 seed 로 +1.81%, p=4.01e-05 를 얻었을 때 "우연 아닌 가?" 의심이 있었음. **5 seed 반복으로 CI 하한이 양수로 확정되어 real effect 임**이 통계적으로 증명됨.
- **"평균 개선 +1.64%"의 의미:** BERN baseline 대비 STRAT(KM20) 의 median Q-error 가 1.64% 작다는 뜻. 절대값이 작아 보이지만 **이미 좋은 시스템(Exqutor BERNOULLI) 위에서의 추가 개선**이라는 점이 중요.
- **95% CI [+1.11, +2.18]:** 이 구간이 0 을 포함하지 않으므로 "effect=0 이다" 가설을 기각. 하한이 양수인 것이 핵심.
- **s=5%, 1% 는 왜 CI 없나:** 초기 Phase 6 단일 측정 산출물. 5-seed 재측정은 중간발표 후 W5 에 SIFT mid-sel 과 함께 수행 예정. 그래도 gradient 일관성과 §4.8 RANDOM20 대조에서 상호 검증됨.

표 3 — External Validity (SIFT 1.5M, 2배 효과)

selectivity	KM20 5-seed 평균	95% CI	유의한 seed
50%	+3.07%	[+2.66, +3.48]	5/5 (p<1e-8)
5%	+4.39%	[+2.63, +6.15]	5/5
1%	-0.53%	[-3.18, +2.11]	1/5

- **"효과 2배"의 정확한 의미:** DEEP 1M 의 s=50% 에서 +1.64%, SIFT 1.5M 의 s=50% 에서 +3.07%. 두 CI [+1.11, +2.18] 과 [+2.66, +3.48] 이 **겹치지 않음** — 통계적으로 구별되는 차이. "우연히 큰 값" 이 아니라 "SIFT 에서 실제로 2배 큰 효과" 임이 CI 분리로 증명.
- **왜 SIFT 에서 2배인가:** SIFT 가 DEEP 보다 쓸림이 심함. CV 0.234 → 0.394 로 **68% 더 쓸림** (표 5 참고). 쓸림이 클수록 공간 인식 샘플링의 이점이 큼 — 연구 가설이 정량 지표 대 효과 크기의 단조 관계로 뒷받침됨.
- **s=1% 가 음수인 건 문제인가:** 아님. SIFT s=1% 면 true_card 가 1.5만 정도인데 sample_size=385 에서 매칭 이 거의 안 나와 hook_est 가 양자화(5 seed 모두 BERN median 1.4435 로 동일). 이건 **boundary condition** 의 증거이지 KM20 의 실패가 아님. 뒤의 RANDOM20 대조(-12.11% 악화)에서 KM20 이 방어 역할을 하고 있음이 확인.

- **발표 포인트:** "데이터가 더 쓸릴수록 본 방법의 이점이 커진다" 는 주장을 HHI/CV 정량 지표와 Q-error 개선 폭의 일관된 단조 관계로 증명.

표 4 — KM20 vs RANDOM20 대조 (DEEP 1M, 공간 인식 인과 증명)

selectivity	KM20 (CI)	RANDOM20 (CI)	격차 (Level 2)
50%	+1.64% [+1.11, +2.18]	+2.20%	~0
30%	+2.62% [+1.45, +3.80]	+0.26% [-0.52, +1.05]	+2.4%p
10%	+4.19% [+0.72, +7.66]	+1.74% [+0.50, +2.99]	+2.5%p
5%	+1.85%	+0.79%	+1.1%p
1%	+8.93%	-10.67%	+19.6%p

- 이 표가 본 연구의 가장 강한 증거: "개선의 원인이 뭐냐"는 질문에 대한 답.
- "20 등분 자체가 도움"이라면 KM20 과 RANDOM20 이 비슷해야 함.
- 하지만 s=1% 에서 KM20 은 +8.93% 개선, RANDOM20 은 **-10.67% 악화** — 방향 자체가 반대.
- 격차 19.6%p 는 **공간 구조를 아는 파티션만의 이점**.
- **왜 s=50%에서는 RANDOM20 이 오히려 높은가:** s=50% 면 결과 행이 모든 클러스터에 고르게 분포 → 파티션 품질 불필요. 두 파티션 모두 Level 1(비례 배분) 효과만 작동하고 Level 2(공간 인식)는 거의 0 (Anomaly 1, 보고서 §4.10).
- **왜 s=1%에서 RANDOM20 이 악화되는가:** 좁은 쿼리의 결과 행은 소수 클러스터에 집중됨. 무작위 20등분에서는 원래 집중돼 있던 클러스터의 행이 20개 그룹에 흩어지고, 비례 배분을 해도 매칭 없는 그룹에서 가짜 0 을 많이 세게 됨 → 오히려 악화. KM20 은 집중 클러스터를 정확히 포착하므로 이 함정을 피함.
- Slide 11 의 핵심 시각화: [figure_7_selectivity_gradient.png](#) (gradient 그래프) + [figure_9_two_level_decomposition.png](#) (Level 1/2 분해).

표 5 — HHI / CV 로 본 쏠림 정량 (왜 SIFT 가 더 쏠렸나)

지표	DEEP 1M	SIFT 1.5M	균일 기준
K	20	20	20
최소 클러스터	26,343	33,330	50,000 / 75,000
최대 클러스터	81,233	148,202	50,000 / 75,000
최대/최소 비율	3.08배	4.45배	1.00
HHI	0.05274	0.05776	0.05000
CV	0.234	0.394 (68%↑)	0.000

- **이 표의 역할:** 표 3 의 "SIFT 에서 효과 2배" 를 **인과적으로 설명**. 쏠림이 실제로 더 크기 때문에 공간 인식의 이점도 더 크다는 논리.
- **HHI = 0.05 의미:** 완전 균일일 때 각 클러스터가 5% → $1/K = 0.05$. 관찰값 $0.05274 / 0.05776$ 이 0.05 보다 크다는 건 쏠림의 존재 증거. SIFT 가 DEEP 보다 상대적으로 더 쏠림.
- **CV 0.394 의미:** 클러스터 크기의 표준편차가 평균의 39.4% — 편차가 크다. 특히 SIFT 최대 클러스터 14.8만은 균일 기대값 7.5만의 **2배**. 상위 두 클러스터가 전체의 19% (균일 10% 대비 1.9배).
- **이 숫자들이 Q-error 로 연결되는 경로:** 쏠린 클러스터에서 정확한 비례 표본을 뽑으려면 파티션이 그 클러스터를 "안다"는 정보가 필요 → KM20 만 가진 정보 → Level 2(공간 인식) 기여로 발현.

§5. 보고서 논리 흐름 — 왜 이 순서인가

중간보고서 [속도는벡터_중간보고서_20260417_0000.md](#) 의 §2 ~ §5 는 다음과 같은 **논증 사슬**로 엮여있습니다. 읽을 때 이 흐름을 따라가면 각 절이 왜 필요한지 보입니다.

§2 (필요성) — "왜 이 문제가 문제인가"

- §2.1: 벡터 DB 카디널리티 추정이 고정 비율이라 잘못된 실행 계획 문제. Exqutor 해법 소개.
- §2.2: Exqutor 의 Adaptive Sampling 은 "무작위 샘플 = 전체 대표" 가정 위에 있음 → 쏠린 데이터에서 깨짐. 4/3 교수님 합의.

역할: 독자가 "이 연구가 해결하려는 진짜 문제가 뭔지" 납득하게 만들. Exqutor 가 해결한 것과 해결 못한 것의 경계.

§3 (연구 질문) — RQ1 / RQ2 / RQ3

- RQ1(motivation): Exqutor 단일 테이블 사각지대가 정말 있는가, 어떻게 측정 가능한가
- RQ2(aware): 분포를 알 때 층화 샘플링이 얼마나 개선하는가
- RQ3(agnostic): 분포를 모를 때 얼마나 회수할 수 있는가

역할: 4/3 3단계 프레이밍을 학술 RQ 로 엄밀히 바꿈.

§4 (연구 내용) — 날짜순 서사

§4.1~§4.10 이 **4/3 합의** → **4/14~17 측정 결과** 순으로 배열됨. 이게 독자에게 어떻게 보이냐면:

- §4.1: "우리는 4/3 에 이런 합의를 했다" (출발점)
- §4.2: "그 합의의 H1 을 검증해봤더니 기각되었다" (반전 1)
- §4.3: "같은 날 Exqutor design constraint 를 발견했다" (반전 2)
- §4.4: "두 발견 위에서 Pivot A+C 병합 노선을 확정했다" (재설계)
- §4.5: "Pivot A 를 Native 로 검증했다" — Phase 4 (§4 표 1)
- §4.6: "Pivot C 를 Native 로 구현했다" — Phase 6 Step 4 + 5-seed (§4 표 2)
- §4.7: "외적 확장을 시도하다 Phase 7 artifact 를 발견했다" (솔직한 자기 검증)
- §4.8: "D_target 재계산 후 SIFT/8M 에서 재측정에 성공했다" (§4 표 3)
- §4.8 계속: "KM20 vs RANDOM20 대조로 공간 인식 인과를 증명했다" (§4 표 4)
- §4.9: "HHI/CV 로 쓸림을 정량 비교해서 SIFT 2배 효과의 원인을 설명했다" (§4 표 5)
- §4.10: "Two-Level Decomposition 으로 개선의 기여를 수학적으로 분해했다"

역할: 독자가 "이 팀이 어떻게 문제를 발견하고 풀어갔는지" 를 **실제 시간 순서**로 따라오게 함. Phase 7 artifact 를 숨기지 않고 서술하는 건 측정 엄밀성의 증거 (reviewer 에게 플러스).

§5 (종합 — RQ 진행률과 한계)

- 5.1: RQ 별 진행률 (RQ1/2 완료, RQ3 설계만)
- 5.2: **한계 4가지** — (a) 단일 dataset family, (b) SIFT mid-sel CI 결손, (c) boundary $s \geq 0.050$, (d) `vector.c` hook 수정의 reviewer 위험
- 5.3: 4/28 이후 W5~W8 로드맵

역할: 자신감과 솔직함의 균형. 한계를 명시해서 자문 회신 여지를 열어두되, 각 한계에 해소 계획이 붙어 있어 "그래서 안 끝난 게 아니다"를 보임.

읽기 팁

- 표의 모든 숫자는 `experiments/results/` 안 JSON/log 에 원본이 있음. 의심되면 원본부터 확인.

- figure 들은 [experiments/figures/rq1_motivation/](#) 과 [rq2_aware/](#) 에 있음. 발표 전 경로 확인 필수.
- Phase 7 서술(§4.7) 이 "이게 왜 있지?" 싶을 수 있는데, 이건 reviewer 가 "왜 SIFT/8M 데이터가 늦게 나왔나" 물을 때의 답. 초기 측정 artifact 를 솔직히 드러내고 해소 경로를 보인 것.

§6. RQ3 설계안 개요 — 4/28 이후 실험 계획

RQ3 는 중간발표 시점에서는 **설계만** **확정**된 상태이고, 실험은 4/29~5/18 에 수행합니다. 설계안 원본: [plans/RQ3설계안_20260416_213500.md](#) .

RQ3 가 묻는 것

"데이터 분포를 사전에 모를 때(예: 실시간 INSERT, 새 데이터셋), KM20 의 공간 인식 효과를 얼마나 회수할 수 있는가?"

KM20 은 분포를 알고 k-means 를 돌려야 해서 실무 한계 4가지: 1. 사전 계산 필요 (1M: ~30초, 8M: 수 분) 2. stratum_id 저장 (모든 행에 컬럼 추가) 3. 데이터 변경 시 재계산 4. K 선택 문제

이걸 극복하는 방법 7가지를 체계적으로 비교.

Recovery Rate 프레임워크

$$\text{recovery_rate} = (\text{방법X} - \text{RANDOM20}) / (\text{KM20} - \text{RANDOM20})$$

- 1.0 = KM20 과 동등 (oracle 수준)
- 0.0 = RANDOM20 과 동등 (공간 인식 없음)
- 음수 = RANDOM20 보다도 나쁨

이 지표 자체가 연구 기여. "절대 Q-error 크기" 대신 "oracle 대비 회수율" 로 보면 비교가 명료해짐.

3 패러다임 × 7 방법

패러다임 1: Offline Partition (데이터 로드 시 1회 계산) — 4개

방법	원리	기대 Recovery	구현 난이도
C. Random Projection	JL lemma, 랜덤 1D 사영 후 K등분	10~40%	하
A. LSH	랜덤 하이퍼플레인 p개 → 2^p 버킷	30~60%	중
E. Hilbert Curve	d차원 → 1D 공간 보존 곡선	20~60%	중
F. Mini-batch K-means	1~5% 데이터만 보고 근사 KM	75~95%	하

패러다임 2: Online Query-Adaptive (쿼리 시점 동적) — 2개

방법	원리	기대 Recovery
G. Distance-Shell	쿼리 중심 동심원 shell 분할	25~50%
B. KDE-pilot	Silverman KDE + Neyman 최적 배분	50~80%

패러다임 3: Weight-based (파티션 없이 가중치) — 1개

방법	원리	기대 Recovery
H. Importance Sampling	밀도 역수 가중치로 보정	30~70%

예상 순서

$\text{RANDOM20} \leq C \leq \{A, E, G\} \leq \{B, H\} \leq F \leq \text{KM20}$

F(Mini-batch KM)가 가장 높을 것으로 예상 — 근사 k-means 이므로. Hilbert Curve(E)는 공간 DB 에서 30년 검증된 기법을 벡터 카디널리티 추정에 **최초 적용** 하는 것이 기여.

구현 통찰 (재현 용이성)

7가지 방법의 차이는 **stratum_id 할당 방식** 뿐. 측정 파이프라인(multiseed_paired, D_target numpy, 로그 파싱, paired Wilcoxon)은 RQ2 완성분을 그대로 재활용. 각 방법마다:

1. Python 으로 stratum_id 계산 (50~100줄)
2. DB에 `UPDATE table SET stratum_id = 새값`
3. 기존 파이프라인으로 측정

H(Importance Sampling) 만 hook_est 반환 공식을 가중 평균으로 수정해야 함. 나머지 6개는 동일.

주별 계획

- W1 (4/28~5/4): Offline 4종 (C/A/E/F) 구현 + 실험 (구현 ~11시간, 실험 ~8시간)
- W2 (5/5~5/11): Online + Weight 3종 (G/B/H) (구현 ~16시간, 실험 ~10시간)
- W3 (5/12~5/18): Recovery Rate 분석 + 패러다임 간 비교 + 비용-효과
- W4 (5/19~5/27): 최종 보고서 + 슬라이드 + 포스터

탈락 후보들 (왜 제외했는가)

24개 후보를 검토해서 17개 탈락. 자세한 건 설계안 §탈락 후보 상세. 간단히:

- D (Exquitor Hybrid): SIGSEGV + negative finding + 범위 밖
- I/K: H/C 의 변형 → 독립성 없음
- LHS/Tree/Balanced: 구현 극난 or 프레임워크 불일치
- Spectral/Optimal Transport: 1M 규모 비현실적
- MAB: 반복 쿼리 전제, 우리 세팅 부적합

§7. 예상 Q&A — 발표장/리뷰 대비

15개. 발표 Slide 별 스크립트에도 일부 녹아있지만, 팀원이 질문받을 때를 대비해 별도로 정리.

Q1. "왜 KM20 인가요? 다른 K 는 안 해봤나요?"

Phase 6 Step 3' 에서 4 layer (bernoulli / pca_decile / kmeans_k10 / kmeans_k20) 를 동시 비교했고 KM20 이 가장 넓은 selectivity 적용 범위를 가졌습니다. K=30/40/50 sweep 은 W5(4/28~) 에서 수행 예정. K 선택 자체가 RQ3 Recovery Rate 의 비교 축 중 하나이기도 합니다.

Q2. "per-stratum ORDER BY random() LIMIT 비용은?"

BERN 대비 약 **11배**. 최종발표 W5 에서 per-stratum **TABLESAMPLE BERNOULLI** 최적화로 2배 이내 감축 목표 (270k 추정 budget 을 75~150시간 → ~25시간으로 축소).

Q3. "SIFT 효과 2배가 정말 쓸림 때문인가요? 다른 요인은?"

CV 68% 격차와 효과 2배 격차 사이의 단조 관계가 증거. 다른 잠재 요인 검토: - 차원 (96d vs 128d): 거리 집중 현상은 차원이 높을수록 심해지니 오히려 SIFT 에서 변별력이 더 약해야 함 → 관측과 반대. - 데이터 크기 (1M vs 1.5M): DEEP 8M 에서도 +1.76% 로 CONSISTENT → 크기 효과가 아님. - 따라서 **쓸림(HHI/CV)** 이 가장 설명력 있는 요인.

Q4. "Phase 7 artifact 를 발견한 계기는?"

3축 심층 디리뷰에서 plan_rows 분포의 이상(STRAT 100/100 에서 정확히 20, std=0) 을 지적 → parquet 직접 조회로 확정. 그 뒤 `find_scan_node` 가 첫 Subquery Scan 의 plan_rows 를 반환하면 garbage 이므로 `hook_est` 만 사용하는 규약을 확정.

Q5. "vector.c 를 수정했으면 Exquitor 가 아닌 다른 시스템 아닌가요?"

맞습니다. 본 팀 측정은 `if (table_count > 2)` → `>= 1` 및 `SYSTEM → BERNOULLI` 수정 상태에서 수행됨. 단 **BERN** 과 **STRAT** 비교는 같은 수정 상태 위에서 이뤄지므로 상대값 비교는 유효. 변경된 `vector.c` 와 원본 백업을 모두 보존 중. 이걸 보고서 §5.2(d) 에 명시된 한계이며 자문 회신 후 narrative 조정 예정.

Q6. "s=0.001 에서 KM20 과 BERN 이 같은 이유는?"

sample 385 안에 매칭이 0~1개 뽑혀서 BERN 의 `cnt=0 → 1` clamp fallback 과 STRAT 의 `cnt_total=0 → est_total=total/budget` fallback 이 같은 값($993541/385 \approx 2.581$) 반환. 이걸 sampling method 가 아니라 sample_size=385 구조적 한계의 결과. 이를 통해 **안정 하한 $s \geq 0.050$** 을 정량 확정한 것이 negative finding 의 학술적 기여.

Q7. "5-seed 이 통계적으로 충분한가요?"

최소한도. 각 seed 가 100 query × paired Wilcoxon 내부에서 이미 충분한 검정력을 가지고, 5 seed 는 seed-to-seed 변동률 봐서 "단일 seed 수치가 우연 아닌지" 를 체크하는 용도. 정밀한 비교가 필요한 RQ3 최종에서는 10 seed × 500 query × 6 selectivity × 3 mode × 3 dataset = 270,000 측정 으로 확대 예정 (W6).

Q8. "왜 Fisher γ 같은 query-side skewness 지표는 안 통했나요?"

100 query 전부 Fisher $\gamma < 0$ (left-skewed) 이고 tail ratio P99/P50 이 1.07~1.17 의 좁은 범위에 모여있어 지표 변별력이 없었음. 96차원 고차원 거리 집중 현상(distance concentration) — 모든 쿼리가 비슷한 분포를 만듦 — 때문. 로컬 4지표까지 검증해서 8지표 전수 부정. 이 negative result 자체가 "query-side 가 아닌 data-side 에서 layer 를 가져와야 한다"는 Pivot C 의 정당화로 편입됨.

Q9. "중간발표 시점에서 RQ3 실험은 왜 안 했나요?"

RQ3 가 체계적으로 하려면 Offline/Online/Weight 세 패러다임 7가지를 모두 공정 비교해야 하고, 이게 약 6주 작업 (구현 ~27시간 + 실험 ~18시간 + 분석). 4/28 마감 전에 부분만 돌리면 오히려 "일부 방법만 유리하게 골라 보여준다" 는 인상을 줄 수 있음. 중간발표는 설계안까지, 실험은 최종발표(5/27) 에서 7-way 전면 비교 후 Recovery Rate 프레임워크로 제시.

Q10. "Exqutor 논문과의 차이는?"

Exqutor 는 multi-table join with vector range filter 를 본선 시나리오로 가정하고 Adaptive Sampling 을 설계. 우리는 단일 테이블 vector range query 사각지대를 공략. 시나리오가 다르고(실무적으로 더 중요), 개선 속이 다름 (Exqutor 의 Adaptive 는 sample_size 조정, 우리는 sample_allocation 조정). 따라서 우리 연구는 Exqutor 를 대체하지 않고 보완.

Q11. "8M 에서 s=1% 가 -0.71% 인데 실패 아닌가요?"

8M 에서 s=1% 면 true_card 가 8만, 그런데 sample_size=385 는 전체의 0.005% 에 불과. Q-error 가 6~8배로 치솟고 seed 별 결과가 -29%~+11.5% 로 극단 흔들림(CI [-21.13, +19.70] 로 0 포함). 이걸 sample_size=385 가 $s \geq 0.050$ 전제로 설계된 결과 → 어떤 샘플링도 이 영역에선 부정확. KM20 의 실패가 아니라 Exqutor 기본 sample_size 구조의 한계. 보고서 Anomaly 2 에 서술.

Q12. "Hilbert Curve 를 왜 쓰나요? 96d 에서 통할까요?"

공간 DB(R-tree) 에서 30년간 검증된 locality 보존 기법이라는 점이 장점. 단 96~128d 고차원에서는 curse of dimensionality 로 locality 보존이 약해질 수 있음 — 이게 negative finding 으로 나오면 그것도 기여 ("Hilbert Curve 가 벡터 카디널리티 추정에는 효과적이지 않다" 는 학술적 발견). RQ3 Recovery Rate 프레임워크가 positive/negative 모두 의미를 살림.

Q13. "RANDOM20 이 오히려 더 나을 수도 있는 상황은?"

s=50% DEEP 1M 에서 RANDOM20(+2.20%) > KM20(+1.64%) 로 관찰됨 (Anomaly 1). 단 두 CI 가 겹치므로 통계적 차이 없음 (seed 간 분산 std ~0.5~1% 안). 이걸 "넓은 쿼리에서는 파티션 품질 무관" 이라는 Level 1/2 분해의 교과서적 예시.

Q14. "자문 회신이 안 오면 어떻게 하나요?"

중간발표·중간보고서는 현재 버전(v2, 4/17 완성) 으로 제출. 자문 회신이 오면 §2~§4 의 narrative 톤만 조정하는 것이 기본 결정. [vector.c](#) hook 우회 수정의 reviewer 관점 위험(§5.2 d) 에 대한 판단이 회신의 핵심 포인트. 박 세은 팀장이 자문 관련 Q&A 를 대응.

Q15. "발표 10분 안에 어떻게 다 말하나요?"

Slide 별 시간 배분이 사전 확정 (발표 md 슬라이드 제작 메모): - S1(0:15) + S2(0:20) + S3(0:35) + S4(0:25) + S5(0:50) + S6(1:00) + S7(0:50) + S8(0:55) + S9(1:10) + S10(0:55) + S11(1:00) + S12(0:35) + S13(0:40) + S14(0:30) = 9:40 - Q&A 5분 여유 - 강재현이 주 발표자로 전체 커버. 리허설 2회 이상 (4/24~4/26) 예정.

§8. 파일 지도 — 어디서 뭘 찾는지

제출 관련 ([submission/](#))

파일	용도
속도는벡터_중간보고서_20260417_0000.{md,pdf,docx}	중간보고서 v2 (4/28 제출본)
속도는벡터_중간발표_20260417_0000.{md,pdf,docx,pptx}	발표 슬라이드 4종 (강재현 주 발표)
속도는벡터_2차 자문요청.{md,pdf}	4/15 송부한 자문 v3
속도는벡터_수행계획서.{hwp,pdf}	4/1 제출본
속도는벡터_연구제안서.docx	4/2 제출본
속도는벡터_연구지도확인서{1~6}.pdf	확인서 이력 6건
archive/	v1(4/15) 버전 보관

계획 문서 ([plans/](#))

파일	내용
연구제안서_20260403_162818.{md,pdf,docx}	4/3 합의 버전 (출발점)
연구재설계안_20260415_131400.md	4/15 v4→v5 재설계 (RANDOM20 control 추가)
RQ3 설계안.{md,pdf}	4/16 확정 7-way 설계
archive/	구버전 보관

실험 결과 ([experiments/](#))

```
experiments/
├─ code/rq1/rq2_run_final.py      RQ2 실행 스크립트
├─ config/                        실험 파라미터
├─ results/
│  ├─ RQ1_RQ2 실험 결과 정리.md    * 모든 수치 원본 통합본
│  ├─ RQ1_RQ2 실험 결과 정리.pdf
│  ├─ rq1_motivation/             Phase 1~5 + artifact 검증
│  │  ├─ summary.md
│  │  ├─ direction_pivot_rationale.md (187줄, 4/14 피벗 근거)
│  │  ├─ phase4_*.parquet          (Pivot A 원본)
│  │  ├─ phase7_artifact_verification_20260415.md
│  │  └─ ...
│  ├─ rq2_aware/                  5-seed CI + cross-dataset
│  │  ├─ anomaly_analysis.md      (Anomaly 3건 해설)
│  │  ├─ cluster_distribution.md  (HHI/CV 상세)
│  │  ├─ sift_1m_mid_summary.json (* SIFT 5-seed 원본)
│  │  ├─ 8m_km20_s0500_5seed.json (* 8M 5-seed 원본)
│  │  └─ sift_1m_mid_full.log
│  └─ rq3_agnostic/              (W5~ 채워질 예정)
├─ figures/
│  ├─ rq1_motivation/
│  │  ├─ figure_1_phase4_scatter.png (Slide 8)
│  │  ├─ figure_2_phase6_box.png     (Slide 9)
│  │  ├─ figure_3_phase6_layer_compare.png (부록)
│  │  ├─ figure_5_python_native_table.png (부록)
│  │  └─ figure_6_phase5_heatmap.png  (Slide 5)
│  └─ rq2_aware/                  (v2 신규 4장)
│     ├─ figure_7_selectivity_gradient.png (Slide 11)
│     ├─ figure_8_cross_dataset_bar.png   (Slide 10)
│     ├─ figure_9_two_level_decomposition.png (Slide 11 보조)
│     └─ figure_10_cluster_skew.png      (Slide 10 보조)
```

기록 (records/)

파일	내용
kakaotalk/20260403_*.md	4/3 교수님 미팅 (Skew-Aware 방향 확정)
kakaotalk/20260415_실험 모식도.{md,pdf}	4/15 팀 공유
kakaotalk/20260415_톡방공유 RANDOM20 결과.{md,pdf}	RANDOM20 control 결과
weekly/overnight_2026-04-17.md	4/16~17 overnight 작업 일지 (239줄)
weekly/주간보고_2026-04-04.md	주차별 보고
팀원 온보딩_20260417.md	이 문서

지침 / 스크립트

경로	용도
CLAUDE.md	프로젝트 헌법 (일정, 디렉토리, 규칙)
guideline/	활성 지침 5종 (실험/제출물/PDF/미팅/발표)
scripts/md2pdf.py	md → PDF 변환 (Chrome CDP, Apple SD Gothic Neo)
scripts/md2docx.py	md → docx 변환
scripts/rq2_figures.py	Figure 7~10 생성
scripts/build_midterm_pptx.py	중간발표 PPT 생성 (1046줄)

참고 외부

대상	링크
Exqutor GitHub	https://github.com/BDAI-Research/Exqutor
팀 Notion	https://www.notion.so/306db4d4869b8039affeca0b0fa4d2fa
팀 GitHub	https://github.com/johyunbin/Capstone
캡스톤 홈페이지	https://capstone.cs.yonsei.ac.kr/capstone/

언제 뭘 보면 되는가

- 숫자 의심 시: `experiments/results/RQ1_RQ2 실험 결과 정리.md` (통합본) → 세부는 `rq2_aware/*.json` 원본
- 왜 이 결정을 했지?: `records/kakaotalk/20260403_*.md` (교수님),
`experiments/results/rq1_motivation/direction_pivot_rationale.md` (Pivot 근거)
- Phase 7 얘기가 뭐였지?:
`experiments/results/rq1_motivation/phase7_artifact_verification_20260415.md`
- 발표 스크립트: `submission/속도는벡터_중간발표_20260417_0000.md` 의 각 Slide 아래 "발표자 스크립트"
- RQ3 는 뭐하는 거지?: `plans/RQ3설계안_20260416_213500.md`
- 서버 접속·빌드: `~/.claude/projects/-Users-hyunbin-Capstone/memory/reference_server.md` (Claude 메모리 — 조현빈 개인 파일)

§9. 앞으로의 일정 — 누가 뭘 할 수 있는가

4/18 ~ 4/28 (중간발표 직전 11일)

날짜	작업	책임
4/18~21	PPT 발표자 노트 내재화, 중간보고서 교열	강재현 (주 발표), 조현빈 (교열 지원)
4/18~21	자문 회신 수령 시 §2~§4 톤 조정	박세은 (자문 대응), 조현빈 (반영)
4/22	1차 팀 리뷰 (최소 1회)	전원
4/23~26	리허설 2회 이상, 피드백 반영	강재현 주도, 이동욱 슬라이드 검토
4/27	최종 교열, 파일 점검	박세은 총괄
4/28	★ 제출 + 발표	강재현 발표

각자 기여 가능한 지점

박세은 (팀장) - 4/22 팀 리뷰 진행 주도 — 발표 전체 흐름, 팀원 질문 수령 - 자문 회신 수령 시 내용 해석 및 대응 Q&A 준비 (특히 `vector.c` hook 수정 정당화) - 4/28 제출 총괄 (양식/파일/일정 확인) - 발표장 Q&A 중 연구 전체 방향·배경 관련 질문 백업

강재현 (주 발표자) - 15 슬라이드 스크립트 완전 내재화 (각 슬라이드 시간 배분 대로 리허설) - 4/23~26 리허설 2회 이상 주도 - 예상 Q&A §7 15문 숙지 + 즉답 가능한 수준으로 - 특히 Slide 8~11(핵심 결과) 은 표 숫자 전부 설명 가능해야 함

조현빈 - 이 문서를 포함한 기술 백업 문서 유지·보완 - Q&A 중 실험 수차·파이프라인· `vector.c` 수정 상세 등 기술 질문 백업 - 4/17 아침 commit 기반으로 팀원 피드백 반영한 세부 교정 - RQ3 W5 착수 준비 (구현 환경 정리)

이동욱 - 슬라이드 검토: 오타, 레이아웃, 그림 해상도, 용어 일관성 - 리허설 피드백 제공 (청중 입장에서 "이 부분이 갑자기 이해 안 간다" 지적) - 제출 직전 파일 무결성 점검 (폰트 깨짐, 페이지 잘림 등)

4/28 이후 (최종발표 준비 W5~W8)

RQ3 7-way 실험 + 최종보고서까지. RQ3 설계안이 이미 확정되어 있으므로, 팀원이 일정 참여 가능하다면 다음 역할 분담도 고려 가능:

- C/F (Offline 단순) 구현·실험: 1~2일 작업, 학습 곡선 완만 — 팀원 누구나 가능
- A/E (LSH/Hilbert): 약간의 논문 리딩 필요 — 관심 있는 팀원
- B/G/H (Online/Weight): 상대적으로 깊은 이해 필요 — 조현빈 주도 예상

W5~W8 의 상세한 분담은 4/28 이후 팀 미팅에서 다시 합의.

맺음말 — 이 문서를 어떻게 쓰지

1. **첫 정주행** (오늘~토요일, 약 25분): §0 → §1 → §4 → §7 순으로. 이거면 전체 그림 잡힘.
2. **리뷰 전 보완** (4/22 전): §2 용어, §5 논리 흐름, §8 파일 지도 까지 두 번째 읽기. 표 숫자 외울 필요는 없지만 "어디에 있는지" 는 기억.
3. **발표 리허설** (4/24~26): §7 Q&A 15문을 팀원끼리 주고받는 mock 질의응답. 막히면 보고서 해당 절로 돌아가 보강.
4. **제출 당일** (4/28): §8 파일 지도로 제출 파일 4종(발표 pdf/pptx + 보고서 pdf/docx) 최종 확인.

수치는 원본(`experiments/results/` 와 `submission/`) 이 항상 진실입니다. 이 문서가 원본과 충돌하면 원본을 따르세요. 업데이트 필요하면 조현빈에게 알려주시면 반영하겠습니다.

— 2026-04-17 09:00 KST 작성 (조현빈 + Claude Code)